

LAPORAN PRAKTIKUM

INTERNET OF THINGS II

Jaringan Nirkabel dan Komunikasi HTTP



Oleh:

NAMA : Albertus Satriyo Nugroho
NPM : 25782002
KELAS : 3A
MATKUL : Internet of Things (IOT)

Dosen:

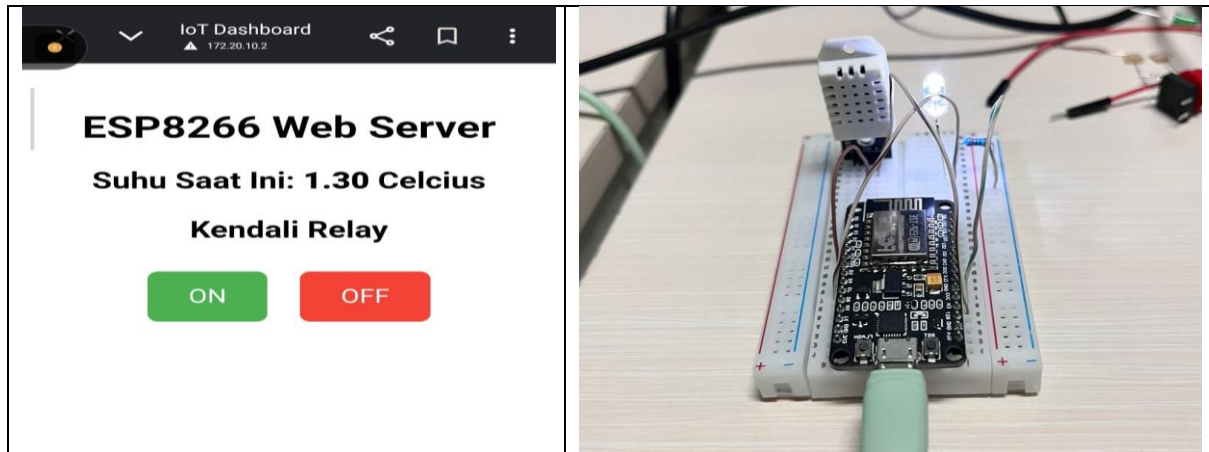
Dr. Ir. Septafiansyah Dwi Putra, S.T., M.T.
Agus Ambarwari, S.Pd., M.Kom.

TEKNOLOGI REKAYASA INTERNET
JURUSAN TEKNOLOGI INFORMASI
POLITEKNIK NEGERI LAMPUNG

2026

3.5.1 Praktikum 1: NodeMCU sebagai *Web Server*

pada praktikum kali ini membuat server pintar untuk membaca sensor DHT 22 dan menampilkan web (UI) sekaligus menyediakan tombol untuk menhidupkan dan mematikan alat module relay, namun karena daya relay tidak mencukupi maka di ganti menggunakan LED , sebagai konfirmasi hidup dan matinya HDT22. Dan untuk masuk kedalam UI kita harus menggunakan jaringan internet yang smartphone kita miliki agar IP dapat di deteksi oleh NodeMCU, sehingga kita bisa mengakses melalui browser.



```
sketch_sep21a.ino
1  #include <ESP8266WiFi.h>
2  #include <ESP8266WebServer.h>
3  #include <DHT.h>
4
5  const char* ssid = "dpinaans";
6  const char* password = "12345678";
7
8  ESP8266WebServer server(80);
9
10 const byte dhtPin = 2;
11 const byte relayPin = 12;
12 DHT dht(dhtPin, DHT11);
13
14 const char index_html[] PROGMEM = R"rawliteral(
15 <!DOCTYPE html>
16 <html>
17 <head>
```

Output Serial Monitor X

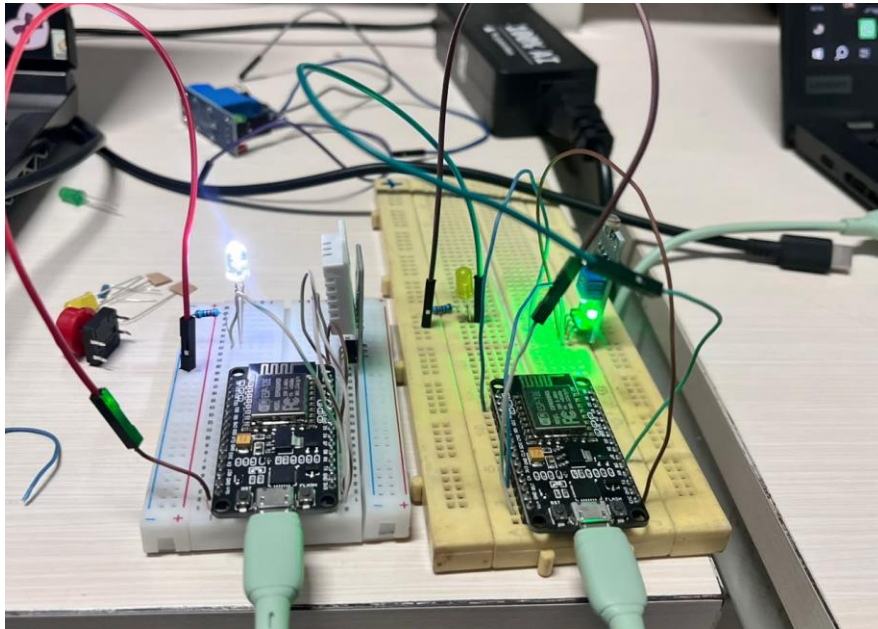
Message (Enter to send message to 'NodeMCU 1.0 (ESP-12E Module)' on 'COM4')

.....

IP Address Server Anda:
172.20.10.2

3.5.2 Praktikum 2: NodeMCU sebagai *HTTP Client*

Praktikum 2 ini kita melakukan HTTP Client dan di sini mengerjaknya berkelompok 2 orang satu sebagai server dan satu sebagai client. Dengan cara memasuka IP yang ada di server lalu mausukan ke code client agar bisa saling terhubung.



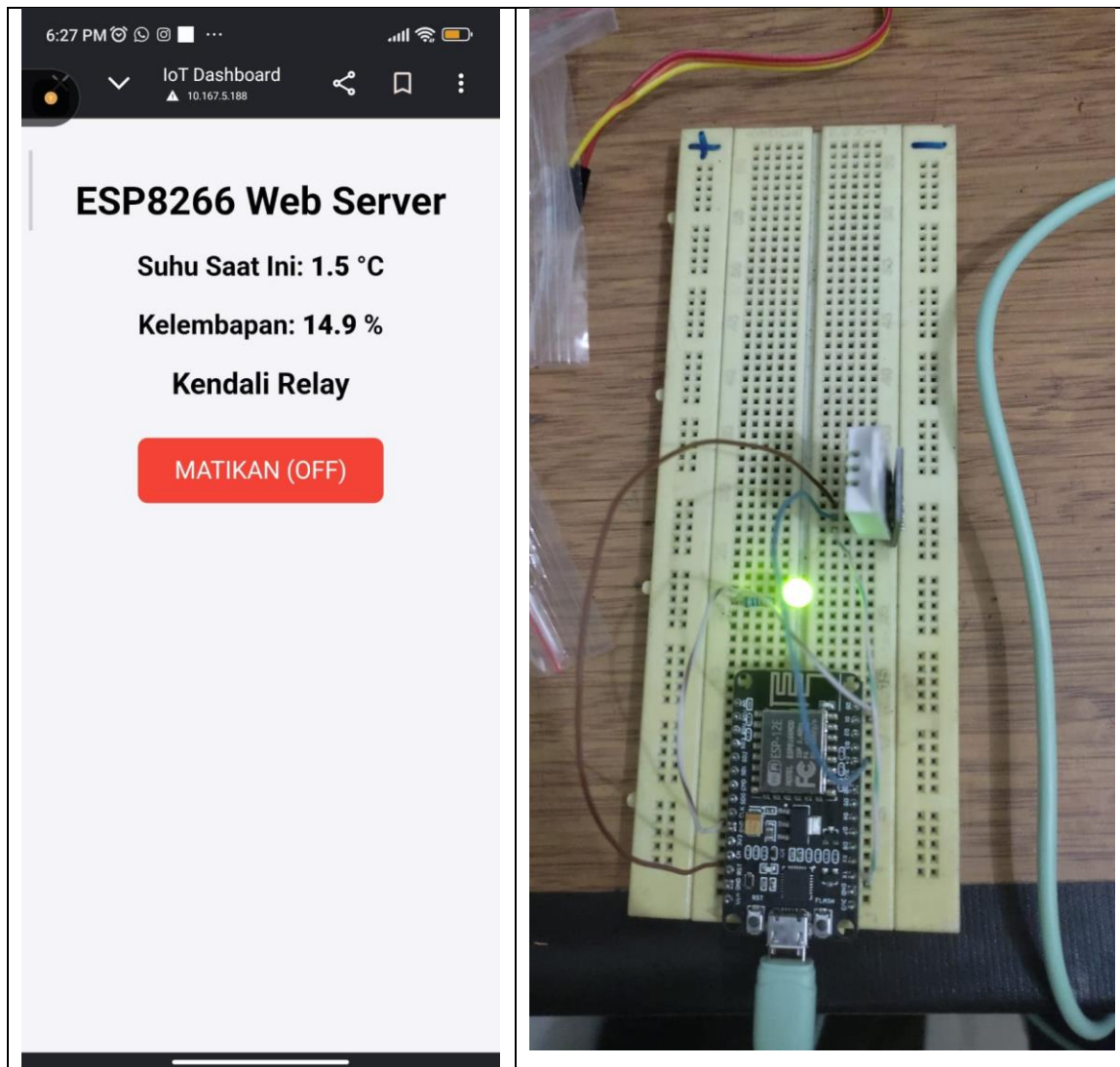
Sebelah kiri Adalah server dan yang kiri client, Ketika client (LDR) Ketika LDR mendeteksi gelap >8000 lampu akan otomatis menyala, dan kita dapat juga memantau melalui dasboard kita juga dapat melakukan control on/off, namun di sini karena menggunakan sensor LDR lampu akan otomatis menyala Kembali.

3.6 Latihan

Latihan 1 Berdasarkan wawasan dari subbab 3.3.1, sebutkan secara logis mengapa kita menggunakan mode jaringan *Station (STA)* pada kedua praktikum di atas, dan bukan mode *Access Point (AP)*?

- Pada mode **STA (Station)**, NodeMCU bergabung sebagai klien ke dalam jaringan Wi-Fi lokal (router/hotspot) yang sudah ada. Hal ini memungkinkan laptop, smartphone, dan perangkat lain yang berada di jaringan yang sama untuk mengakses Web Server NodeMCU secara bersamaan tanpa perlu berpindah koneksi Wi-Fi.
- Jika menggunakan mode **AP (Access Point)**, NodeMCU akan memancarkan sinyal Wi-Fi sendiri. Perangkat user harus memutuskan koneksi internet utamanya hanya untuk terhubung ke Wi-Fi NodeMCU, yang sangat tidak efisien untuk integrasi sistem IoT skala luas.

Latihan 2 Pada Praktikum 1, tampilan web saat ini baru menyajikan data suhu ruangan dan masih menggunakan dua tombol terpisah (ON dan OFF).



Ini hasil praktikum yang saya lakukan tampak dasboart memiliki output yang sesuai dengan interuksi soal, Menambahkan telemetri **Kelembapan** (%HUMIDITY%).

Mengubah tombol dua arah (ON & OFF terpisah) menjadi **Tombol Dinamis Tunggal (Toggle Button)** menggunakan placeholder %RELAY_BUTTON%.

Latihan 3 Pada Praktikum 2, *Serial Monitor Client* mencetak HTTP Response code. Analisislah apa yang akan dicetak oleh *Serial Monitor* Anda jika ternyata NodeMCU *Server* teman Anda mendadak kehilangan daya (*offline*)? (Silakan cabut daya *Server* dan lihat hasil erornya).

Ketika NodeMCU Client mencoba melakukan request HTTP GET ke IP NodeMCU *Server* yang sedang mati/hilang daya (*offline*):

1. Kode HTTP Response yang Dicetak:

Library ESP8266HTTPClient akan mengembalikan nilai angka negatif, spesifiknya **-1** (yang didefinisikan sebagai HTTPC_ERROR_CONNECTION_REFUSED).

1. Pesan Penjelasan pada Laporan:

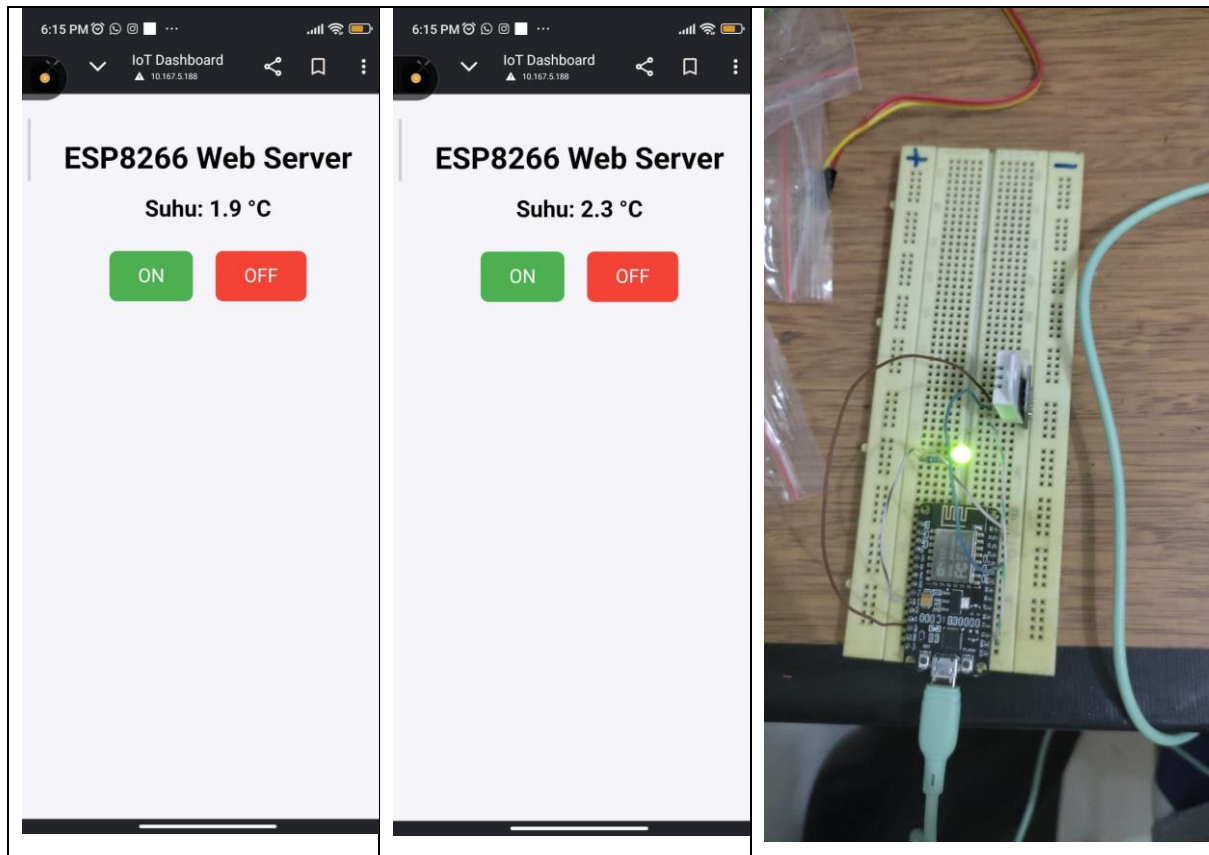
Karena server *offline*, jabat tangan koneksi TCP (*TCP handshake*) pada port 80 gagal dilakukan. Library HTTP Client tidak bisa membuka socket ke alamat IP target, sehingga fungsi `http.GET()` mengembalikan nilai error integer negatif (-1), bukan status kode HTTP standar (seperti 200, 404, atau 500).

Latihan 4 Anda ingin mendesain Klien Pendeteksi Suhu. Modifikasi *conditional block* if di Praktikum 2 sehingga *Client* mengirim permintaan (GET Request) **HANYA JIKA** sensor DHT membaca suhu melebihi 35 Celcius (bukan menggunakan LDR). Tulis baris algoritmanya.

Langkah ini diterapkan pada **NodeMCU Client** untuk mengecek kondisi suhu lokal dari sensor DHT22 sebelum memutuskan untuk mengirimkan HTTP GET Request ke Server.

<code>void loop() {</code>
<code> // Membaca suhu dari sensor DHT lokal</code>
<code> float temp = dht.readTemperature();</code>
<code> </code>
<code> // Validasi pembacaan sensor</code>
<code> if (isnan(temp)) {</code>
<code> Serial.println("Gagal membaca dari sensor DHT!");</code>
<code> delay(2000);</code>
<code> return;</code>
<code> }</code>
<code> </code>
<code> Serial.print("Suhu Lokal Terbaca: ");</code>
<code> Serial.print(temp);</code>
<code> Serial.println(" °C");</code>
<code> </code>
<code> // ALGORITMA UTAMA: Kirim permintaan HTTP HANYA JIKA suhu > 35°C</code>
<code> if (temp > 35.0) {</code>
<code> if (WiFi.status() == WL_CONNECTED) {</code>
<code> WiFiClient client;</code>
<code> HTTPClient http;</code>
<code> </code>
<code> Serial.println("Suhu melebihi 35°C! Mengirim peringatan ke server...");</code>
<code> </code>
<code> // Inisialisasi HTTP GET Request ke IP Server</code>
<code> http.begin(client, "http://192.168.1.50/relay/on"); // Ganti IP sesuai IP Server temanmu</code>
<code> </code>
<code> int httpCode = http.GET(); // Kirim permintaan</code>
<code> </code>
<code> if (httpCode > 0) {</code>
<code> Serial.printf("HTTP Response Code: %d\n", httpCode);</code>
<code> } else {</code>
<code> Serial.printf("Koneksi gagal, error: %s\n", http.errorToString(httpCode).c_str());</code>
<code> }</code>
<code> </code>
<code> http.end(); // Tutup koneksi</code>
<code> }</code>
<code> } else {</code>
<code> Serial.println("Suhu masih aman (<= 35°C). Tidak ada request yang dikirim.");</code>
<code> }</code>
<code> </code>
<code> delay(5000); // Tunda 5 detik sebelum pemeriksaan berikutnya</code>
<code>}</code>

3.7 Tugas: Pengembangan *Web Server* Cerdas dan Analisis Arsitektur



Ini hasil dari tugas saya di sini Nampak sudah realtime untuk update suhunya Dimana terdapat perbedaan di foto 1 dan 2 . Setiap 5 detik, browser menghancurkan seluruh DOM (struktur halaman) dan mengunduh ulang seluruh file HTML. Hal ini menyebabkan layar seolah-olah "berkedip" (flicker) putih tiba-tiba, Jika tombol **ON/OFF** diklik tepat pada detik ke-5 saat browser sedang mengirim permintaan *auto-refresh*, tombol akan berpotensi me-refresh ulang halaman sebelum perintah tombol selesai diproses server.